ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Tipos de Datos y Estructuras de Control

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

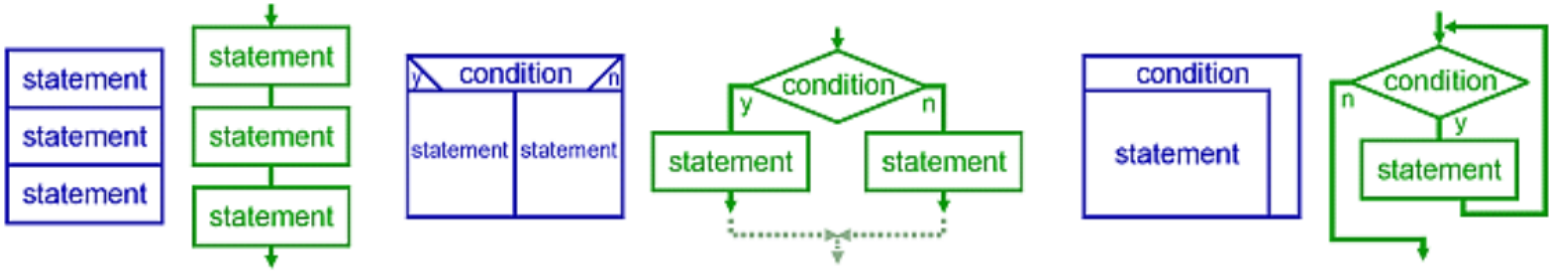
2do Semestre 2022

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Programación Estructurada

Paradigma de programación basado en el **uso extensivo de subrutinas y estructuras de control**, con el fin de mejorar la **claridad**, **calidad** y **tiempo de desarrollo** del código de programas computacionales.

- **C, lanzado en 1973, es el mayor exponente de este tipo de lenguajes.**



¿Por qué es C tan relevante?

- Diseñado para desarrollar UNIX OS
- Maneja actividades de bajo nivel (en la memoria), a pesar de ser un lenguaje de alto nivel.
- Se relaciona eficientemente con instrucciones de máquina.
- Perdura su uso en el tiempo.
- La mayoria de software tiene una relación directa o indirecta con C.

Elementos Generales - Datos

- **Datos Primitivos:** enteros, reales, caracteres.

C

```c
int contador = 10;
char nombre[10] = "Eduardo";
float PI = 3.14159;
```

Python

```python
contador = 10
nombre = "Eduardo"
PI = 3.14159
```

- **Estructurados:** agrupar/componer conjuntos de datos del mismo tipo en una unidad (e.g. Arreglo).

C

```c
char semana[5][20] =
{"Lu","Ma","Mi","Ju","Vi"};
printf("%s\n", semana[1]);
```

Python

```python
semana = ["Lu","Ma","Mi",
          "Ju","Vi"]
print(semana[1])
```

Elementos Generales - Estructuras de Control

- **Sentencias:** abstrae conjunto de instrucciones.
- **Estructuras de control:** secuenciación, condición y repetición (e.g. if, while).
- **Abstracción procedural:** procedimiento con un nombre y parámetros (e.g. funciones y métodos).
- **Concurrencia:** computación paralela (e.g. procesos y hebras).

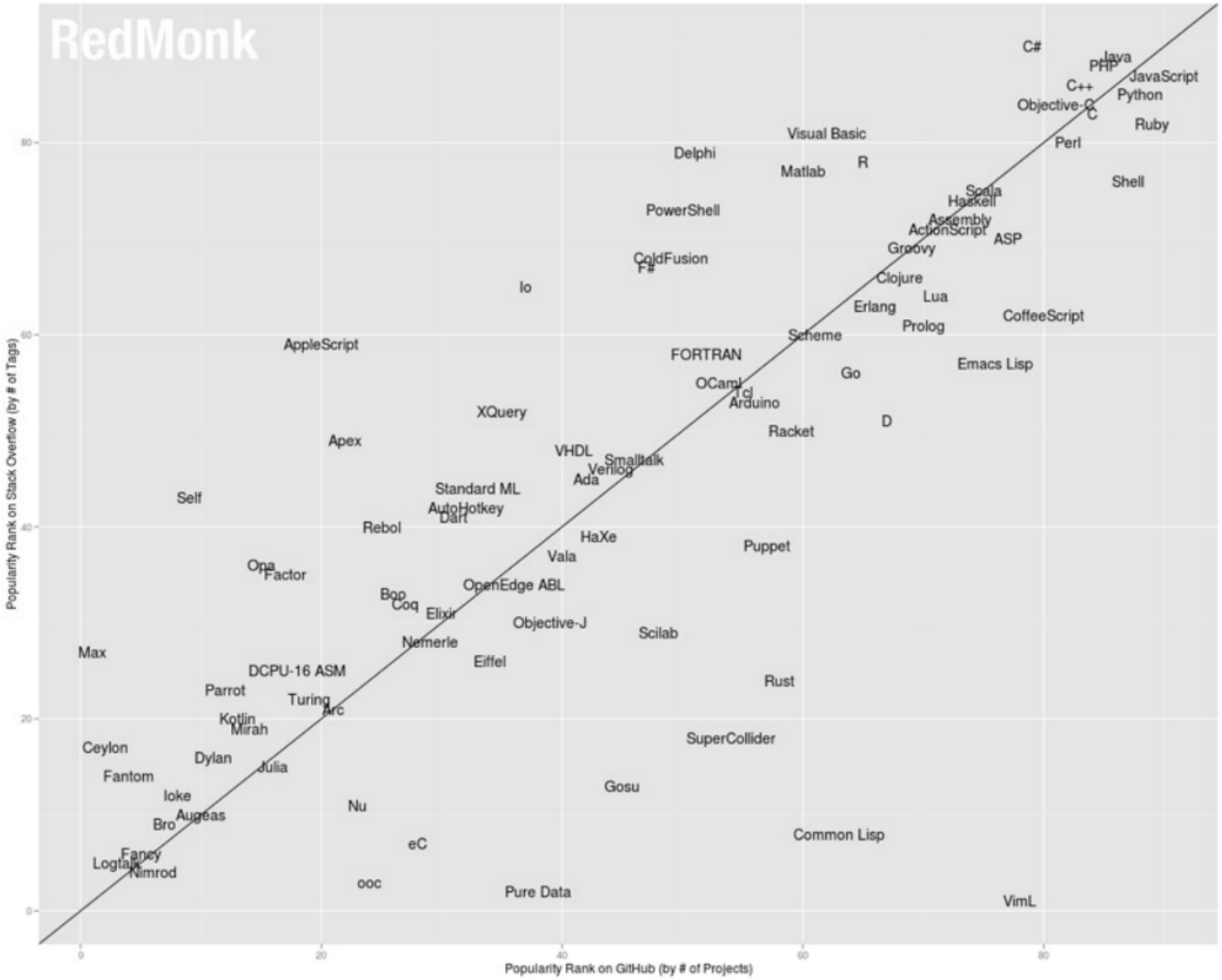
Métodos de Implantación

- **Compilación:** (e.g. C, C++)
  Traduce a lenguaje de máquina para posterior ejecución (interpretación directa por el procesador).
- **Interpretación:** (e.g. LISP, Python)
  Una máquina virtual interpreta directamente el código fuente durante la ejecución.
- **Híbrido:** (e.g. Java, C#)
  Se compila a un lenguaje intermedio, que luego es interpretado.

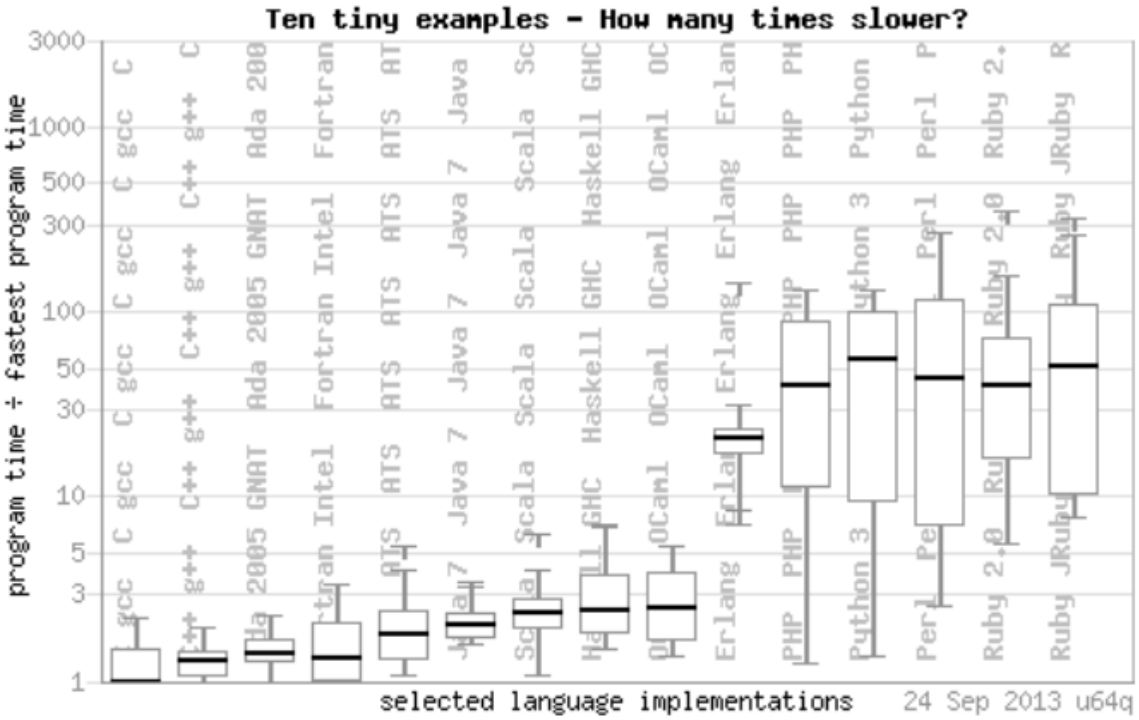
Uso de Código en C

- **Modularización:** descomponer programas en unidades de desarrollo más pequeñas (módulos).
- **Compilación Separada o Indepediente:** Módulos de un programa se pueden compilar por separado (bibliotecas).
- **Reutilización:** Módulos y clases se pueden reutilizar para diferentes programas.

Popularidad de Lenguajes (stackoverflow, redmonk)



Rendimiento de Lenguajes



http://benchmarksgame.alioth.debian.org/u32/performance.php

C Estructura general

- Requiere un método de inicio de ejecución: **main**

C

```c
#include<stdio.h>

int main() {
        printf("Hola Mundo\n");
        return 0;
}
```

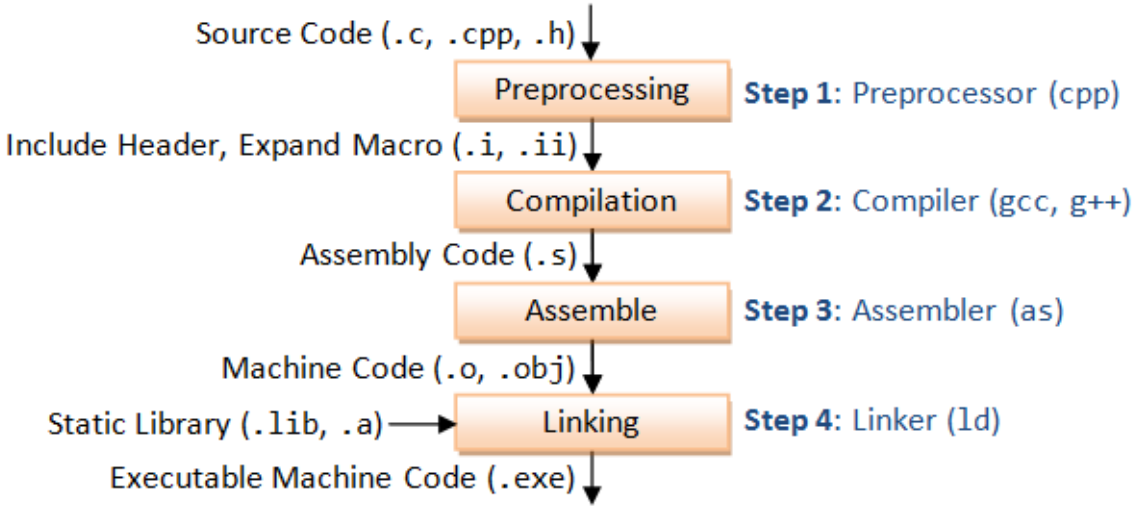
Python

```python
if __name__ == "__main__":
        print("Hola Mundo!")
```

Compilación: gcc

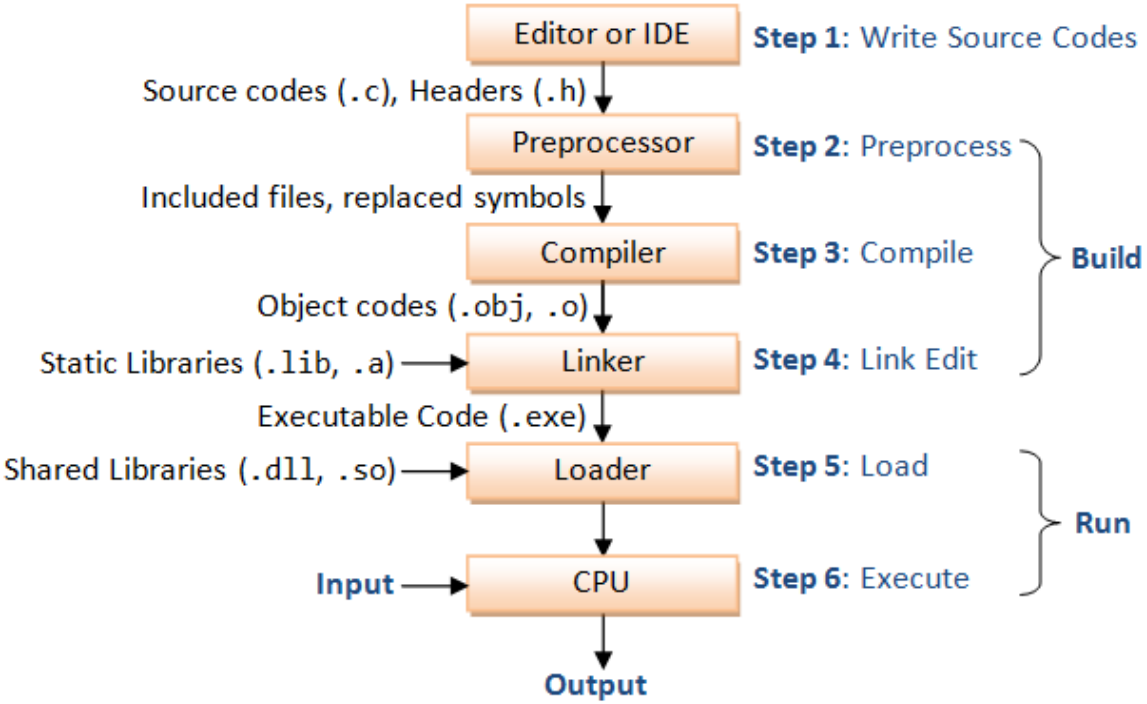
C es un lenguaje compilado, esto significa que:

- Su código es transformado, a través de un compilador, a lenguaje máquina.
- El resultado de la compilación es un binario ejecutable.
- Binarios ejecutables: .bin, .exe, .appinstall, etc.



Compilación: gcc

Quizas la imagen anterior ahora tenga mayor sentido:



Compilación: gcc

Compliación directa

```
[user@pc dir]$ gcc -o ejecutable codigo1.c codigo2.c -Wall
```

donde:

- **-o** (output) indica que lo siguiente es el nombre del ejecutable.
- **codigoX.c** es el archivo de nuestro código (pueden ser más de uno)
- **-Wall** indica que el compilador de aviso de potenciales peligros (warnings)

Compilación: gcc

Compilación por pasos

```
[user@pc dir]$ gcc -c codigo1.c -Wall
[user@pc dir]$ gcc -c codigo2.c -Wall
[user@pc dir]$ gcc -o ejecutable codigo1.o codigo2.o
```

donde:

- **-c** indica que un archivo.c será compilado independientemente a un archivo objeto .o.
- **-o** (output) indica que lo siguiente es el nombre del ejecutable.
- **-Wall** indica que el compilador de aviso de potenciales peligros (warnings)
- En la última instrucción se enlazan todos los archivos compilados de forma independiente.

Tipificación de Datos - Explícita VS Implícita

- Exigencia en la declaración para detectar potenciales errores de tipo.
- **Tipificación explícita (C):**
  - Siempre detecta errores de tipo.
  - Restricciones sobre operaciones y valores de diferentes tipos de datos.
  - No hay ejecución si tipos son erróneos.
  - Tipificación estática y explícita.
- **Tipificación implícita (Python):**
  - Conversiones que permiten relajar restricciones.
  - Flexibilidad y agilidad.
  - Genera errores no detectados.

Tipos de datos en C

- Se define qué tipo de dato es cada variable antes de usarlas (tipificación explícita):

C

```c
#include<stdio.h>

int main() {
        int a;
        float d;
        int b = 5, c = 6;
        printf("%d\n", b+c);
        return 0;
}
```

- En Python los tipos de datos se infieren de la asignación (tipificación implícita).

Tipos de datos primitivos en C

Descripción	C	Python
Booleano	bool	bool
Byte	char	int
Caracter	char	str
Entero corto	short	int
Entero	int	int
Entero largo	long	long
Punto flotante	float	float
Pto. flotante largo	double	float
String	-	str
Complejos	-	complex

Tipos de datos primitivos en C

tipo	bytes	Rango
char	1	-127 to 127 or 0 to 255
unsigned char	1	0 to 255
signed char	1	-127 to 127
int	4	-2147483648 to 2147483647
unsigned int	4	0 to 4294967295
signed int	4	-2147483648 to 2147483647
short int	2	-32768 to 32767
unsigned short int	2	0 to 65,535
signed short int	2	-32768 to 32767
long int	4	-2,147,483,647 to 2,147,483,647
signed long int	4	same as long int
unsigned long int	4	0 to 4,294,967,295
float	4	+/- 3.4e +/- 38 (7 digits)
double	8	+/- 1.7e +/- 308 (15 digits)
long double	8	+/- 1.7e +/- 308 (15 digits)

Tipos de Datos

- **Sistema de tipificación en C:**
  - **Verificación de tipo:** Los operandos de una operacion deben ser de *tipo compatible*.
  - **Tipo compatible:** es un tipo permitido, o que puede ser convertido en uno permitido.
  - **Conversión de tipos:** *Coerción* es conversión automática y *casting* es la conversión definida por el programador.

```
int pi = 3.14;              //coerción
double d = (double) pi;     //casting
```

  - **Error de tipo:** aplicación de un tipo inapropiado en una instrucción.

Especificadores

Las funciónes de I/O de C utilizan especificadores para identificar el tipo de dato a leer o imprimir:

Tipo	Especificador
char	%c
int	%d %i
unsigned int	%u
float	%f %.3f
string	%s
double	%lf
long int	%ld
long long unsigned int	%Lu

Entrada Estándar: scanf

Función de entrada estándar, se compone de una sección de especificadores y otra sección de variables a asignar el valor leído.

scanf

```
int a;
char c;
char nom[10];
double prec;

scanf("%d", &a); //lee un entero
scanf("%c", &c); //lee un char
scanf("%s", nom); //lee un string

//lee un double, un string y un entero.
scanf("%lf %s %d", &prec, nom, &a);
```

Salida Estándar: printf

Función de salida estándar, se compone de una sección de especificadores y otra sección de variables cuyos valores se mostrarán.

printf

```
printf("%d", a); //imprime un int
printf("%c", c); //imprime un char
printf("%s", nom); //imprime un string

//imprime un double de 3 decimales, un string y un entero.
printf("%.3lf %s %d", prec, nom, a);
```

Definir ≠ Asignar ≠ Inicializar

C

```c
#include<stdio.h>
int main(){
        int a; //Define a la variable a como tipo entero
        a = 3 + 2; //Evalúa 3+2 que es 5 y se asigna a 'a'.
        printf("%d\n", a);
        return 0;
}
```

En C soporta el uso de ambos conceptos (definición y asignación) en conjunto. A esto se llama inicialización.

C

```c
#include<stdio.h>
int main(){
        int a = 3 + 2;
        printf("%d\n", a); // Inicializa con resultado 3+2
        return 0;
}
```

Enumeración

C

```c
enum color {
        rojo,
        amarillo,
        verde=20,
        azul};

int col = rojo;

if (col != azul){
        printf("No es azul\n");}
```

Python (clases)

```python
class Colors(Enum):
        red = 1
        yellow = 2
        green = 3
```

Preview Próx. Clases: Arreglos y Strings

- **Arreglo:** Tipo estructurado consistente en conjunto homogéneo y ordenado de elementos.
- Se identifican por su posición relativa mediante un *índice* encerrado con paréntesis cuadrados (parte de índice 0).

```
//Arreglos estáticos (se liberan al terminar
// el ámbito donde fueron declaradas):
int foo[10]; //10 enteros creados en foo.
foo[3] = 7; //A cuarto elemento se asigna un 7.
```

- **String:** Arreglo de caracteres.

```
// Los string se delimitan con caracter '\0'
// (requiere espacio de 1 char):
char nombre[10] = "Juan"; //String de largo máximo 10.
nombre[1] = 'e'; //Qué hace?
printf("%s\n", nombre);
```

- Veremos más sobre arreglos la próxima clase.

Expresiones en Asignaciones

- **Valor de una expresión** determinado por **reglas de orden de evaluación de operadores y operandos**:
  - Asociatividad.
  - Precedencia.
- Si **orden es ambigüo** puede conducir a **diferentes resultados**, **según la implementación** del traductor de lenguaje.
- Tip: Usar paréntesis ante la duda.

Expresiones Aritméticas

Consisten en conjunto de **operadores**, **operandos** (argumentos), **paréntesis** y **llamadas a funciones**. El **orden de evaluación** (precedencia, asociatividad y paréntesis) determina su **valor**.

- **Aridad de operadores:**
  - *Unario (unitario):* Un sólo operando.
  - *Binario:* Dos operandos.
  - *Ternario:* Tres operandos.

Operadores Unarios

Operador	**C**	**Python**
Negación lógica	!	not
Dirección	&	*No existe*
Derreferencia	*	*No existe*
Incremento	++	*No existe*
Decremento	- -	*No existe*
Signo más	+	+
Negativo	-	-
Casting	(tipo)	*No existe*

Los operadores unarios agrupan de derecha a izquierda.
E.g. $-+x$ equivalente a $-(+x)$.

Operadores Binarios

Operador	C	Python
Suma	+	+
Resta	-	-
Multiplicación	*	*
División	/	/
División entera	/	//
Módulo	%	%
Exponenciación	*No existe*	**

División entera en C según tipo de dato.

Si tipo entero para numerador y denominador: división entera.

Orden de Evaluación de Operadores

- **Precedencia:** Orden de evaluación de operadores adyacentes, estableciendo prioridades.
- **Asociatividad:** A igual precedencia, define orden de evaluación.
- **Paréntesis:** Altera y fuerza orden de evaluación.

Precedencia	**Descripción**	**Operador**		**Asociatividad**
		C	**Python**	
alta	exponente		**	←
	unarios (postfijos)	++ −−		→
	unarios (prefijos)	++ −− ∼ ! * & + -	∼ + -	←
	casting de tipo	(type)		←
	multiplicativos	* / %	* / // %	→
	aditivos	+ −	+ −	→
baja	NOT lógico		**not**	←

Ojo con conversiones!

- Expresión puede tener muchos tipos distintos.
- Entonces, el lenguaje hace conversión implícita (coerción), para usar el operador adecuado.
- Mayoría de lenguajes usa conversión por expansión.
- **Pro:** Da más flexibilidad a uso de operadores.
- **Contra:** Reduce la capacidad de detectar algunos errores e introduce código adicional.
- Control **implícito** de conversión mediante *coerción*:

```
float    fVal;
double   dVal;
int    iVal;
unsigned long ulVal;

dVal = iVal * ulVal; //iVal es convertido a unsigned long.
// El resultado es convertido a double

dVal = ulVal + fVal; //ulVal convertido a float.
//El resultado es convertido a double
```

Sentencias de asignación

- Permiten cambiar dinámicamente el valor ligado a una variable.
- C permite **incrustar una asignación en una expresión** (actúa como cualquier operador binario).
- Existen diversos operadores que integran operadores binarios con asignación. Ejemplo:

```
a *= 5;
//Equivalente a:
a = a * 5;
// Todos siguen misma estructura.
```

Sentencias de Asignación

Operador	**C**	**Python**
Asignación	$=$	$=$
Suma y asignación	$+=$	$+=$
Resta y asignación	$-=$	$-=$
Multiplicación y asignación	$*=$	$*=$
División y asignación	$/=$	$/=$
Módulo y asignación	$\%=$	$\%=$
Incremento	$++$	
Decremento	$--$	
División entera y asignación		$//=$
Exponente y asignación		$**=$

Unarios de Asignación - C:

```
//pre-incremental
sum = ++contador;

//equivale a:

contador = contador + 1;
sum = contador;
```

```
//post-incremental
sum = contador++;

//equivale a:
aux = contador;
contador = contador + 1;
sum = aux;
```

Operadores Compuestos - C/Python:

```
sum += a;

//equivale a:
sum = sum + a;
```

Asignación múltiple simple.

C

```c
int a, b, c, d;
a = b = c = d = 5;
```

Python

```python
a = b = c = d = 5
```

Asignación múltiple compuesta.

C

```c
int a = 10, b, c = 15;
a += b = c -= 5;
```

Python (lista)

```python
x, y, z = 20, 30, 40
```

- **C: Único operador ternario.**

```
retorno = op1 ? op2 : op3;
```

Si op1 es verdadero, asigna op2; sino, asigna op3.

```
cuenta = (a > 0) ? 0 : 1;

//Equivale a:
if (a > 0)
        cuenta = 0;
else cuenta = 1;
```

- **Python:**
  retorno = op2 **if** op1 **else** op3

Asignación en expresiones

- C considera la asignación como un operador más.
- Permite incrustar asignación en cualquier expresión.
- **C** Permitido.

```
while ( (c = getchar()) != EOF) {}
```

- **Python:** Sintaxis no lo permite.

```
if (d = 5 + c) > 6:
```

Error: Invalid syntax.

- Pro: Programación más compacta.
- Contra: Fuente de error en la programación. Es fácil equivocarse confundiendo == e =.

Expresiones Relacionales y Lógicas

- **Expresión Relacional:**
  - Usan operador relacional (binario) para comparar los valores de dos operandos.
  - Cada operando puede ser una **expresión aritmética**.
  - El **valor resultante** de la expresión es **booleano**.
  - Incluye: $==$, $!=$, $<$, $<=$, $>$, $>=$
- **Expresión Booleana:**
  - Consiste en variables, constantes y operadores booleanos.
  - Se puede combinar con expresiones relacionales.
  - Incluye: *AND*, *OR*, *NOT* y, a veces, *XOR*.
- **El único valor falso en C es el 0 (cero).**

Expresiones Relacionales y Lógicas

Operadores Relacionales		
Operador	**C**	**Python**
Igual	==	==
Distinto	! =	! = <>
Mayor	>	>
Menor	<	<
Mayor o igual	>=	>=
Menor o igual	<=	<=
Operadores Booleanos		
Operador	**C**	**Python**
NOT	!	**not**
AND	&&	**and**
OR	||	**or**

Estructuras de control

- C/Python la ejecución es típicamente secuencial.
- Dos tipos básicos de estructuras control:
  - **Condicional:** múltiples alternativas de ejecución, controladas por una condición.
  - **Iterativa:** Ejecución repetitiva de un grupo de sentencias, también controladas por una condición.
- Se requiere mecanismo de agrupación de sentencias sobre las cuales se ejerce el control: **composición de sentencias**.
- En **C** se usa { }
- **Python** compone por indentación.

if - else if - else

C

```
if(condición_1) {
    //....
} else if(condición_k) {
    //....
} else {
    //....
}
```

Python

```
if condición_1:
    #....
elif condición_k:
    #....
else:
    #....
```

switch

C

```
switch(x){ //Debe ser una variable
case x1:
        //....
        break;
case x2:
        //....
        break;
//....
case xN:
        //....
        break;
default: //opcional
        //....
}
```

No existe en Python.

Ciclos controlados por condicion: *while, do-while*

C

```
while(condición) {
    //...
}

do {
    //...
} while(condición);
```

Python:

```
while condición:
    #...
```

Ciclos controlados por contador: *for*

C

```
//En general:
for(sentencia_inicial;condición;sentencia_fin_ciclo) {
    //...
}
```

C

```
//Uso corriente
for(i=0; i<N; i++) {
    //...
}

//Equivalente a:
i=0;
while(i<N) {
    //...
    i++;
}
```

Python:

```
//En general:
for var_iteracion in secuencia:
    #instrucciones ...
```

```
for letra in 'Py':      # Ejemplo 1
   print 'Letra actual:', letra

frutas = ['platano', 'manzana', 'mango']
for fruta in frutas:        # Ejemplo 2
   print 'Fruta actual:', fruta
```

```
Letra actual: P
Letra actual: y
Fruta actual: platano
Fruta actual: manzana
Fruta actual: mango
```

Corto Circuito o Término anticipado de evaluación

- **Corto circuito:** Evaluación de una expresión termina antes.
- **C** definen corto circuito para && y ||.
- Lenguajes como **Perl y Ruby** evalúan todos sus operandos en una expresión lógica.

```
//Si primera condición se cumple, la expresion es true
if((a >= 0) || (b < 10))

//Si primera condición no se cumple, la expresion es false
while ( (c = getchar()) != EOF && c != '\n' )
```

- Puede generar inconvenientes en expresiones con efectos colaterales.

Saltos incondicionales

- **C/Python:**
  - **break:** Permite salir del ciclo más cercano.
  - **continue:** Permite pasar al ciclo siguiente.
  - Se recomienda usarlos con precaución y critero.

Saltos incondicionales - Ejemplo

```
int main() {
  int n;
  do {
    printf("Ingrese numero: ");
    scanf("%d", &n);
    if (n < 0)
      break;
    if (n >10) {
      printf("Saltarse el valor.\n");
      continue;
    }
    printf("El numero es: %d\n",  n);
  } while (n != 0);

  return 0;
}
```